Reviewer Pack — Matroid Rank Gap in Monotone DNF Depth for k-CLIQUE

Entry bfe3aa3f84d1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 11:42:34 UTC

Matroid Rank Gap in Monotone DNF Depth for k-CLIQUE

Entry ID: `bfe3aa3f84d1` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-08 11:42:34 UTC

1. Conjecture

Field A (mathematical branch): matroid theory **Field B** (complexity object): monotone circuit lower bounds for k-CLIQUE

Statement:

> Let μ(f) denote the maximal number of disjoint terms in a DNF representation of a Boolean function f. For any monotone DNF formula of size poly(n), μ(f) = O(log n). For the k-CLIQUE indicator function, μ(f) = Ω(n^{1/2})

Rationale (proposer's reasoning):

> Matroid rank functions are inherently submodular, and their growth rate under conjunction directly relates to the structural complexity of DNF representations. The k-CLIQUE function's exponential DNF complexity forces μ(f) to scale with √n, while polynomial-size DNFs can be compressed into logarithmic rank via matroid decomposition techniques.

Taxonomy category: `BOUNDED_ARITHMETIC` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `77beb4cb1c8a0053`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_monotone_dnf(n, size):
        terms = []
        for _ in range(size):
            term = [random.choice([0, 1]) for _ in range(n)]
            if all(term[i] == 0 or term[j] == 0 for i, j in combinations(range(n), 2)):
                terms.append(term)
        return terms

    def is_independent_set(S, dnf_formula):
        n = len(dnf_formula[0])
        edges = [(i, j) for i in range(n) for j in range(i + 1, n)]
        for u, v in edges:
            if any(S[i] == S[j] == 1 for term in dnf_formula if term[u] == term[v]):
                return False
        return True

    def compute_mu(dnf_formula):
        max_disjoint_terms = 0
        n = len(dnf_formula[0])
        for i in range(1 << n):
            S = [i >> j & 1 for j in range(n)]
            if is_independent_set(S, dnf_formula):
                max_disjoint_terms += 1
        return max_disjoint_terms

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        size = int(n**2)
        dnf_formula = generate_monotone_dnf(n, size)
        mu_f = compute_mu(dnf_formula)
        results.append({"n": n, "size": size, "mu_f": mu_f})

    mean_mu_f = sum(result["mu_f"] for result in results) / len(results)
```

```
    std_mu_f = math.sqrt(sum((result["mu_f"] - mean_mu_f)**2 for result in results) / len(results)
    support_fraction = all(mu_f <= math.log(n) for n, _, mu_f in results if n < 10)

    return {
        "metric_name": "μ(f)",
        "metric_value": mean_mu_f,
        "instances_tested": len(results),
        "conjecture_holds": support_fraction,
        "counterexample": "" if support_fraction else f"n={results[-1]['n']}, mu_f={results[-1]['m
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_mu_f = sum(result["metric_value"] for result in results) / len(results)
    std_mu_f = math.sqrt(sum((result["metric_value"] - mean_mu_f)**2 for result in results) / len(
    support_fraction = all(result["conjecture_holds"] for result in results if "counterexample" no

    if support_fraction:
        print(f"RESULT: SUPPORTED mean={mean_mu_f} std={std_mu_f} support_fraction=1.0")
    elif any("counterexample" in result for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if "counterexample"
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_81a60df6.py", line 73, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_81a60df6.py", line 52, in run_trial
    mu_f = compute_mu(dnf_formula)
           ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_81a60df6.py", line 42, in compute_mu
    if is_independent_set(S, dnf_formula):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_81a60df6.py", line 33, in is_independen
    if any(S[i] == S[j] == 1 for term in dnf_formula if term[u] == term[v]):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_81a60df6.py", line 33, in <genexpr>
    if any(S[i] == S[j] == 1 for term in dnf_formula if term[u] == term[v]):
```

```
            ^
NameError: name 'i' is not defined. Did you mean: 'id'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to NameError, preventing data collection. Pre-registered support condition cannot be evaluated without successful runs. | next: Fix the NameError in the test script (undefined variable 'i' in line 33) and re-run experiments

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	45911
2	propose	ollama_remote	qwen3:8b	0	0	113997
3	propose	ollama_remote	qwen3:8b	0	0	95365
4	preregistration	ollama_remote	qwen3:8b	0	0	24560
5	novelty	ollama_remote	qwen3:8b	0	0	21021
6	novelty	ollama_remote	qwen3:8b	0	0	15982
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15171
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9149
9	judge	ollama_remote	qwen3:8b	0	0	18287

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 359442 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/bfe3aa3f84d1.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/bfe3aa3f84d1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/bfe3aa3f84d1.tar.gz` (if generated)